

Learning Subgraph-Based Normality for Interpretable Graph-Level Anomaly Detection

Ge Zhang¹, Zhenyu Yang², Jia Wu², Senior Member, IEEE, Pengfei Jiao³, Member, IEEE, Jian Yang⁴, Member, IEEE, Hao Peng⁵, Senior Member, IEEE, and Xixun Lin⁶

Abstract—Graph-level anomaly detection (GLAD) aims to identify graphs that significantly deviate from others in a graph dataset. Existing methods predominantly rely on standard Graph Neural Networks (GNNs) to learn graph representations, but they often overlook subgraph-level information, which provides essential structural and semantic cues for distinguishing normal and anomalous graphs. This limitation not only compromises the detection performance but also hinders the interpretability of GLAD predictions. To address these challenges, we propose NGLAD, a novel framework that introduces the concept of normality-relevant subgraphs that capture shared patterns across normal graphs. These subgraphs serve as key indicators to distinguish normal graphs from anomalies that often lack or deviate from such patterns. During model training, by explicitly modeling the shared subgraph patterns inherent in normal graphs through a Subgraph Extractor and a Normality Learner, NGLAD identifies the subgraphs most relevant to normality. Leveraging the One-class Information Bottleneck principle, these modules ensure that the extracted subgraphs retain only the most informative features of normality while filtering out irrelevant nodes and edges. During inference, NGLAD detects anomalies by evaluating inconsistencies in representations between the input graph and its extracted subgraph. Extensive evaluations on synthetic and real-world datasets demonstrate that NGLAD significantly outperforms state-of-the-art methods in detection performance while offering interpretable explanations.

Index Terms—Graph-level anomaly detection, graph neural networks, explanation.

Received 31 January 2025; revised 31 October 2025; accepted 16 December 2025. Date of publication 26 December 2025; date of current version 29 December 2025. This work was supported in part by Shanghai Pujiang Program under Grant 24PJA004; in part by the Fundamental Research Funds for the Central Universities under Grant 2232025D-33; in part by the National Natural Science Foundation of China under Grant 62402491, Grant 62372146, Grant 62322202, and Grant U25B2029; in part by China Postdoctoral Science Foundation under Grant 2025M771524; and in part by the Key Technology Research and Development Program of the Zhejiang Province under Grant 2025C01023. The associate editor coordinating the review of this article and approving it for publication was Dr. Angelo Spognardi. (Corresponding authors: Pengfei Jiao; Xixun Lin.)

Ge Zhang is with the School of Information and Intelligent Science, Donghua University, Shanghai 201620, China (e-mail: gezhang@dhu.edu.cn).

Zhenyu Yang, Jia Wu, and Jian Yang are with the School of Computing, Macquarie University, Sydney, NSW 2109, Australia (e-mail: zhenyu.yang@mq.edu.au; jia.wu@mq.edu.au; jian.yang@mq.edu.au).

Pengfei Jiao is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: pjiao@hdu.edu.cn).

Hao Peng is with the School of Cyber Science and Technology, Beihang University, Beijing 100191, China (e-mail: penghao@buaa.edu.cn).

Xixun Lin is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China (e-mail: linxixun@iie.ac.cn).

Digital Object Identifier 10.1109/TIFS.2025.3647221

I. INTRODUCTION

GRAPHS are commonly used to model relational data in various fields, including chemistry, finance, social networks, and cybersecurity [1], [2]. The task of detecting anomalies within graphs has garnered significant interest from both academia and industry. Although anomalies are often associated with individual nodes, they can also manifest as atypical graphs within a collection or database of graphs [3], [4]. Graph-level anomaly detection (GLAD) refers to the challenge of identifying graphs that differ significantly from the majority in a given dataset. Identifying these anomalous graphs is crucial for maintaining societal stability and safeguarding business interests, such as uncovering fraud syndicates [5].

Recent advances in GLAD [6], [7], [8] typically assume that the graph representations produced by Graph Neural Networks (GNNs) are adequate to differentiate anomalous graphs from normal ones. These approaches often integrate traditional anomaly detection techniques, such as one-class classification [9] and knowledge distillation [10], with standard message-passing GNNs. However, these methods lack the ability to explain the reasoning behind their predictions, i.e., they do not clarify why a particular graph is considered normal or anomalous. In practical settings, providing clear explanations for anomaly detection is crucial. Explanatory models in GLAD increase trustworthiness and ensure that the models meet safety and security standards [11], [12], [13], [14], [15]. For instance, an explainable intrusion detection system can not only detect intrusions but also identify specific intrusion patterns, making the model's predictions more reliable.

We propose that the key to interpreting GLAD's predictions lies in learning the shared subgraph patterns inherent in normal graphs, which are typically missing or disrupted in anomalous ones. These patterns, referred to as **normality-relevant subgraphs**, serve as primary indicators of normality by encapsulating the shared structural and semantic information within normal graphs. As shown in Fig. 1, the “house”-like subgraph functions as a normality-relevant subgraph in the graph dataset. Specifically, graphs G_a and G_c are identified as normal due to the presence of this “house”-like subgraph, while graphs G_b and G_d are detected as anomalous, as the subgraph-level patterns in these two graphs diverge from the normal pattern. This illustrative example reflects a general phenomenon that can also be observed in real-world domains. For instance, in molecular screening, biologically active compounds often

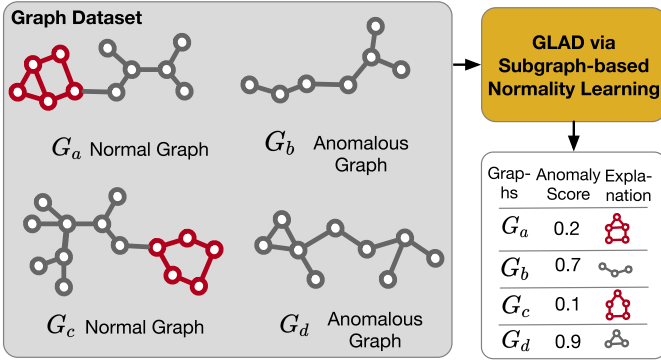


Fig. 1. A toy example about the differences between normal and anomalous graphs. Graphs G_a and G_c are identified as normal with low anomaly scores because they both contain the “house”-like subgraph, which is learned as a normality-relevant subgraphs. In contrast, graphs G_b and G_d are flagged as anomalous with higher anomaly scores, because that it have different local patterns with the normal graphs.

contain functional groups such as carboxyl or hydroxyl groups, which often participate in key interactions contributing to biological activity. In contrast, inactive compounds may lack such groups or possess modifications that disrupt these interaction patterns. In particular, incorporating normality-relevant subgraphs in GLAD not only enhances anomaly detection, but also provides interpretability, enabling the model to explain why specific graphs are identified as normal or anomalous.

Despite its promise, effectively leveraging subgraph-based normality learning for GLAD remains challenging: *How can we identify subgraphs that capture the most representative normality across the graph dataset?* Several recent works have attempted to address this challenge. SIGNET [16] improves the explainability of GLAD by extracting subgraphs through minimizing the multi-view mutual information between the original graph view and its hypergraph view. However, it only considers intra-graph consistency and ignores structural regularities shared across normal graphs. Consequently, the extracted subgraphs may be coherent within individual graphs but fail to represent patterns typical of the entire normal graph population. GLADPro [17] interprets GLAD through learnable prototype representations. Yet, obtaining representative subgraphs that embody the normality of normal graphs is still challenging, as converting prototypes into actual subgraphs requires heuristic procedures.

To address the aforementioned challenges, we propose a novel GLAD method called Self-interpretable Graph-level Anomaly Detection via Subgraph-based Normality Learning (NGLAD for abbreviation). NGLAD explicitly learns normality-relevant subgraphs through the combination of two innovative modules: firstly, a Subgraph Extractor, proposed to identify potential subgraphs from each input graph; Secondly, a Normality Learner enables the model to extract subgraphs that capture patterns shared across normal graphs while faithfully representing their original graphs. We optimize NGLAD using the One-class Information Bottleneck (OIB) principle [18], which combines the Subgraph Extractor and the Normality Learner via a probabilistic framework with a distortion term and a mutual information term. The distortion term

evaluates discrepancies between the extracted subgraphs and the normality of normal graphs, while the mutual information term ensures that only the most informative subgraphs are retained, filtering out irrelevant nodes and edges. Through this probabilistic optimization, NGLAD effectively isolates normality-relevant subgraphs and encodes them for graph-level representations. During inference, NGLAD evaluates the inconsistency between the representations of an input graph and its extracted subgraph to determine abnormality. Normal graphs yield lower anomaly scores, as the model has been trained to extract subgraphs that capture the intrinsic normality of normal graphs. We summarize contributions as follows:

- **Meaningful insight.** We introduce the insight that normality-relevant subgraphs, which capture the most indicative and shared patterns within normal graphs, are essential for improving detection performance and enabling self-interpretability in GLAD.
- **Novel algorithm.** We propose NGLAD, a novel algorithm for detecting graph anomalies through subgraph-based normality learning. By integrating the Subgraph Extractor and Normality Learner modules into the OIB framework, NGLAD captures the normality across graphs and uses these features to distinguish anomalous from normal graphs.
- **Extensive Evaluation.** Extensive experiments on synthetic and real-world graph dataset demonstrate that NGLAD outperforms existing methods in both prediction and interpretation. Visualization experiments highlight the subgraphs identified by NGLAD, effectively explaining the normal or anomalous characteristics of graphs. We also conducted time efficiency experiments to demonstrate that NGLAD is computationally efficient.

II. RELATED WORKS

A. Graph Neural Networks

Graph Neural Networks (GNNs) have shown substantial success in various prediction tasks on graph-structured data, including node and graph classification [19], [20], [21], [22].

1) *Message-Passing Neural Networks:* GNNs that incorporate spatial graph convolution [23], [24], [25] are often discussed within the context of message-passing neural networks (MPNNs) [21], [26], [27], [28]. These MPNNs typically update each node’s representation by iteratively aggregating information from neighboring nodes. To construct a graph-level representation, MPNNs employ a readout function or pooling operation on the node embeddings after the message passing process. The readout function can be as simple as taking the sum, average, or max of all node embeddings in the graph. Alternatively, more sophisticated pooling techniques, such as hierarchical pooling [29], can be used to selectively aggregate node information, potentially giving more weight to certain nodes or regions of the graph. The ability of GNNs to distinguish various graph structures is essential, as it directly impacts their effectiveness in capturing relational patterns and encoding structural information critical to downstream tasks [14], [30], [31], [32]. The Graph Isomorphism Network (GIN)

[26], for example, achieves equivalence to the 1-Weisfeiler-Lehman (1-WL) test [33] by incorporating a multi-layer perceptron (MLP) into its aggregation function. Further, some GNNs leverage subgraphs to capture localized structural patterns, allowing for more expressive graph representations. For example, Zhang and Li [14] proposed to improve upon traditional GNNs by using rooted subgraphs. Specifically, it applies a base GNN to learn node representations based on a subgraph around it, and pools them to form a whole-graph representation, offering superior power compared to the 1-WL algorithm, especially in distinguishing r -regular graphs. In addition, Zhao et al. [15] proposed a framework that enhances the expressiveness of MPNNs by extending local aggregation from star-shaped patterns to general subgraph patterns (e.g., k -ego nets). The framework uses a GNN-based subgraph encoder, offering a more powerful representation than 1-WL and 2-WL, with minimal scalability overhead and improved performance on graph machine learning tasks.

2) *Interpretable Graph Neural Networks*: Since GNNs leverage both node attributes and structural information, their interpretability often centers around identifying influential subgraphs or feature sets that play a crucial role in the model's decision-making process. A wide array of post-hoc explanation methods has been proposed to analyze pre-trained GNNs. These techniques typically work by post-processing a trained GNN and focusing on identifying the parts of the graph that contribute most to the final output. They generally employ combinatorial search techniques to isolate critical substructures within the input graphs, such as attention mechanisms, gradient-based attribution, and subgraph selection methods [34], [35], [36], [37]. In contrast, some studies have proposed inherently interpretable GNN models, where the interpretability is integrated into the model architecture itself rather than relying on post-hoc analysis [13], [38]. For example, GSAT [11] introduces a stochastic attention mechanism that directs the model's focus toward task-relevant subgraphs, thus enhancing the interpretability of the decision process. These inherently interpretable models aim to improve the transparency of the GNN by design, enabling clearer insights into how decisions are made without the need for additional explanation methods.

However, most of the existing research on GNN explainers has focused on supervised learning settings, particularly node or graph classification tasks, which often assume labeled data for training. This presents a significant challenge when attempting to apply these methods to GLAD problems [16], where anomalies need to be identified without explicit labels for abnormal or normal instances. In GLAD, the focus is often on detecting structural deviations or outliers in an unsupervised manner, making it more difficult to interpret the learned representations. Thus, there is a need for explanation techniques tailored to GLAD that can uncover the reasons behind anomaly detection decisions.

B. Graph Anomaly Detection

Graph Anomaly detection focuses on identifying samples (i.e., nodes, edges, subgraphs, and graphs) that significantly deviate from the majority, flagging potential outliers

in the dataset [39], [40]. Various graph anomaly detection algorithms have been developed, often leveraging metrics like distance-based [41], [42], [43] and density-based [44] measures to quantify deviations among samples. In particular, methods such as one-class classification [9], [45] have emerged as prominent approaches. These methods focus on distinguishing normal samples from potential anomalies by learning a decision boundary that encompasses the majority class, effectively identifying outliers that fall outside this boundary.

1) *Graph-Level Anomaly Detection*: Graph anomaly detection has gained significant traction, particularly with GNN-based methods, shifting from a primary focus on detecting anomalous nodes [46], [47], [48], [49] to addressing anomalous graphs, which are crucial for applications like anomalous functional molecule recognition [4] and brain network analysis [50]. For example, Zhao and Akoglu [7] introduced the first GLAD method called OCGIN, which integrates one-class classification with GNNs to identify anomalous graphs. However, they discovered that this type of method often experiences “performance flip”, where the model performs worse than random on at least one experimental variant. To address this issue, Qiu et al. [6] developed a new self-supervised GLAD model named OCGTL, consisting of multiple jointly trained GNNs. Another self-supervised approach, GOOD-D [51], utilizes contrastive learning to capture latent patterns shared by normal graphs. Additionally, the knowledge distillation-based method GLocalKD [8] employs a different mechanism to identify the major patterns in normal graphs. It trains a predictor GNN to represent normal graphs, aligning its outputs with those of another GNN that has randomly initialized and fixed weights.

2) *Interpretable Graph-Level Anomaly Detection*: Few GLAD methods have recognized the significance of subgraphs for the GLAD task; however, they do not specify which types of subgraphs are effective for differentiating between normal and anomalous graphs. For example, iGAD [4] integrates the random walk with GNN to explore anomalous substructures. However, regarding the GLAD problem as the binary graph classification task makes the approach less adaptable to the variations found in anomalous graphs. Another approach, SIGNET [16] introduces the hypergraph as a supplemental view of the original graph and maximizes the mutual information between subgraphs extracted from the supplemental and original views. However, its limitation lies in primarily focusing on each single graph, without effectively capturing the characteristics shared by the entire group of normal graphs. GLADPro [17] proposes to provide global explanations for GLAD by learning several prototype representations that characterize the normal graph population. However, these prototypes exist only as embedding vectors, and mapping them to actual subgraphs relies on heuristic strategies. In addition, determining the appropriate number of prototypes remains a challenging problem. This paper enhances GLAD through subgraph-based normality learning, aiming to capture the most informative local features of each normal graph that contribute to the overall normality of the graph dataset.

III. PRELIMINARIES

A. Graph

A graph can be denoted as $G = (\mathcal{V}_G, \mathcal{E}_G)$, where $\mathcal{V}_G$ is the set of n nodes, $\mathcal{E}_G$ is the set of m edges. The topology of G can be modeled as an adjacency matrix $\mathbf{A}_G \in \{0, 1\}^{n \times n}$, where $\mathbf{A}_G(u_i, u_j) = 1$ if there is an edge between nodes u_i and u_j , i.e., $(u_i, u_j) \in \mathcal{E}_G$, otherwise $\mathbf{A}_G(u_i, u_j) = 0$. $\mathbf{X}_G \in \mathbb{R}^{n \times d}$ is the attribute matrix of G , each row vector $\mathbf{x}_u \in \mathbb{R}^d$ is associated with the attribute vector of node $u_i \in \mathcal{V}_G$.

B. Normality-Relevant Subgraph

A normality-relevant subgraph, denoted as $G^{(S)}$, refers to a specific part of a normal graph G that mostly indicates its normal characteristics. In general, such subgraphs are present in normal graphs but absent in anomalous ones. For example, in identifying mutagenic molecules, functional groups like NH_2 and NO_2 serve as normality-relevant subgraphs, as their presence typically indicates mutagenicity. In contrast, anomalous, non-mutagenic molecules lack these functional groups.

C. Graph Neural Networks (GNNs)

GNN are deep learning models proposed to process graph-structured data by leveraging the relationships between nodes and their neighbors to learn representations at node-level or graph-level. Node feature representations are initialized according to their corresponding attributes, i.e., $\mathbf{h}_{u_i}^{(0)} = \mathbf{x}_{u_i}$. GNNs update the node representation by iteratively aggregating the representations coming from its one-hop neighborhood. The formulation is:

$$\mathbf{h}^{(l)}(u_i) = f_{\text{Combine}}(\mathbf{h}^{(l-1)}(u_i), f_{\text{Aggregate}}(\{\mathbf{h}^{(l-1)}(u_j), u_j \in \mathcal{N}(u_i)\})) \quad (1)$$

where $l \in \{1, \dots, L\}$, $\mathbf{h}^{(l)}(u_i) \in \mathbb{R}^q$ denotes the immediate representation of node u_i at the l -th graph convolution. $\mathcal{N}(u_i)$ denotes the 1-hop neighborhood of node u_i . $f_{\text{Aggregate}}$ denotes how to aggregate neighbors of node u_i , and f_{Combine} defines how to combine the immediate representations of node u_i and its neighborhood. $\mathbf{h}(u_i) = \mathbf{h}^{(L)}(u_i)$ is the final node representation of u_i , where $\mathbf{h}(u_i) \in \mathbb{R}^p$. The graph representation $\mathbf{h}(G) \in \mathbb{R}^p$ is generally obtained via the graph pooling methods applying on $\{\mathbf{h}(u_i), u_i \in \mathcal{V}_G\}$, e.g., sum-based or average-based node representation aggregation.

D. Graph-Level Anomaly Detection (GLAD)

The traditional GLAD framework, represented as $\text{score}(G) = f(G)$, involves identifying anomalous graphs from the graph dataset by learning the anomaly score $\text{score}(G)$ for each input graph G . This score serves as an indicator of the degree of abnormality. In this paper, given the training graph set $\mathcal{G}$ consists of N normal graphs, we aim to train a self-interpretable GLAD model $(G^{(S)}, \text{score}(G)) = f(G)$, where $G^{(S)}$ is the key subgraph extracted from the graph G . If a test graph G having the higher anomaly score, it indicates that G deviates from the normality in the graph dataset, thus being detected as an anomalous graph. Moreover,

the subgraph $G^{(S)}$ provide insights into the reasoning behind the identification of the input test graph as either normal or anomalous.

E. One-Class Information Bottleneck (OIB)

The OIB [18] aims to identify a coherently structured subset of samples from a dataset $X = \{x_1, \dots, x_n\}$ and project this subset into a compact yet informative representation space C . In the context of the one-class problem, this objective is formulated as minimizing a tradeoff between two complementary terms:

$$\min_{\mathbf{w}} [\mathcal{D}(C, \mathbf{w}; X) + I(C; X)]. \quad (2)$$

The first term, $\mathcal{D}(C, \mathbf{w}; X)$, is the distortion term, which measures the closeness between the representation space C and the original data X . Specifically, for a sample x_i assigned to a coherently structured region, a penalty is imposed according to its distance from the region center $\mathbf{w}$ in C . Minimizing this term encourages samples within the same coherent region to cluster together, yielding a compact and consistent representation. The second term, $I(C; X)$, quantifies the mutual information between C and X , ensuring that the representation retains only samples relevant to the coherent structure while excluding those that cannot form tight connections. By jointly minimizing these two terms, the model learns a representation space composed of a subset of samples from X that is both compact and informative, effectively capturing the coherently structured regions while excluding samples that do not belong to these regions.

The OIB principle inspires us to design a mechanism for identifying normality-relevant subgraphs from normal graphs. The rationale is based on the observation that normality-relevant subgraphs form coherent regions, consistent with OIB's objective of identifying coherent subsets from data samples and representing them concisely and informatively. By treating subgraphs within normal graphs as elements of the graph dataset, OIB has the potential to guide the model to focus on subgraphs that capture common structural patterns shared across normal graphs, while filtering out irrelevant information. As a result, the OIB framework provides a principled criterion for selecting subgraphs that effectively capture the inherent normality of the graph collection. However, adapting OIB for interpretable GLAD is a nontrivial endeavor, involving the design of a subgraph extraction mechanism and the requirement that selected subgraphs remain both informative and interpretable, which further demands careful reformulation of the distortion and mutual information terms.

IV. METHODOLOGY

This section provides a detailed overview of the proposed NGLAD model, with its pipeline illustrated in Fig. 2. The methodology begins with the NGLAD framework overview (Sec. IV-A), where we focus on the optimization objective grounded in the OIB principle. Next, we discuss the Subgraph Extractor module (Sec. IV-B), detailing the methodology for extracting subgraphs from input graphs. The Normality

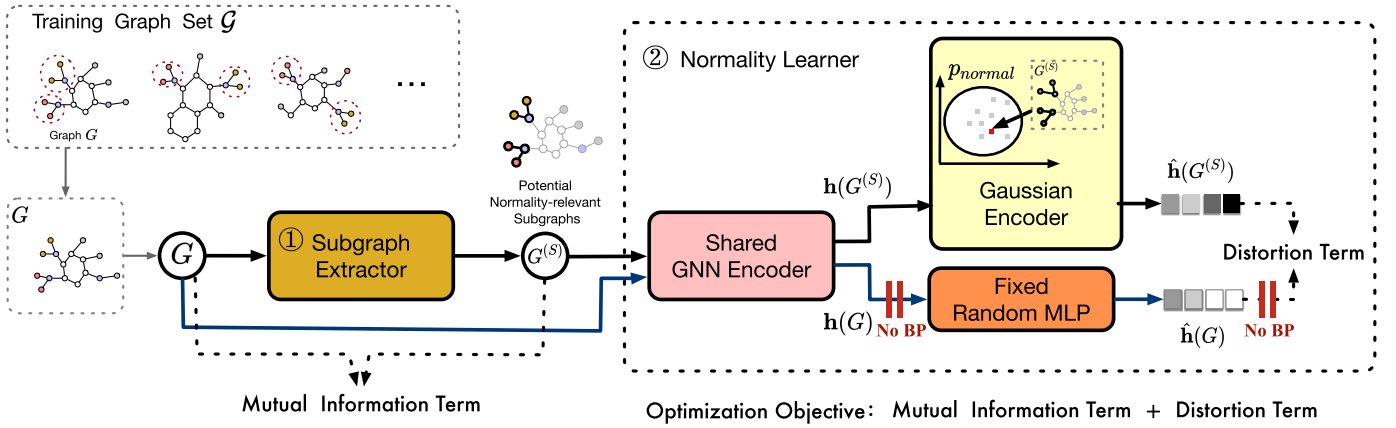


Fig. 2. The overall pipeline of the proposed NGLAD model consists of two key modules: the Subgraph Extractor and the Normality Learner. The Normality Learner includes a Shared GNN Encoder, a Gaussian Encoder, and a Fixed Random MLP. As illustrated in the figure, NGLAD learns normal graph samples by extracting subgraphs relevant to normality. The subgraph within the red dashed circle highlights those that are normality-relevant.

Learner module consisting of a Shared GNN Encoder, a Gaussian Encoder and a Random Fixed MLP (Sec. IV-C) is then described, which learns the alignment between those extracted subgraphs and the normality present in normal graphs. Afterwards, we describe the training process of NGLAD (Sec. IV-D) and explain how NGLAD detects anomalous graphs and interprets GLAD (Sec. IV-E). Finally, we analyze its model complexity (Sec. IV-F).

A. Framework Overview of NGLAD

NGLAD is a novel framework for self-interpretable GLAD via subgraph-based normality learning. It extends OIB from Euclidean space to graph-structured data. To this end, NGLAD introduces the Subgraph Extractor and Normality Learner, which allow the model to identify normality-relevant subgraphs from normal graph data and interpret GLAD predictions. Formally, the optimization objective of NGLAD is defined as:

$$\min_{\Psi, \Phi} \mathcal{D}(\Psi(G; \Theta), \Phi; G) + I(\Psi(G; \Theta); G), \quad (3)$$

where Ψ and Φ denote the Subgraph Extractor and the Normality Learner, respectively. These two modules are jointly designed to identify normality-relevant subgraphs from normal graphs. While Eq. (3) is inherited from the OIB principle, its adaptation in NGLAD necessitates new module designs, including subgraph extraction strategies, graph normality learning, and graph-specific information measures, all of which are specifically tailored for interpretable GLAD. Specifically, NGLAD introduces the following key innovations upon OIB:

(1) **Subgraph extraction for normality modeling:** Unlike OIB, which operates on Euclidean data, NGLAD incorporates a Subgraph Extractor $\Psi(G; \Theta)$ to identify candidate subgraphs $G^{(S)}$ that capture structural patterns shared across normal graphs. This extractor explicitly models complex dependencies between nodes and edges, a challenge that the original OIB framework does not consider.

(2) **Normality-driven distortion evaluation:** The distortion term $\mathcal{D}(\Psi(G; \Theta), \Phi; G)$ is defined with respect to a Normality

Learner Φ , which evaluates how well the extracted subgraphs preserve the inherent normality patterns in the normal graph collection. This formulation generalizes the notion of coherence in OIB from generic data subsets to graph-structured data.

(3) **Graph-specific mutual information measurement:** The mutual information term $I(\Psi(G; \Theta); G)$ quantifies the retained information between the extracted subgraphs $G^{(S)}$ and the original graph G . The computation of mutual information in graph structures presents a significant challenge because of the intricate dependencies among nodes and edges.

By jointly minimizing the newly proposed distortion term and mutual information term, NGLAD identifies normality-relevant subgraphs that ensure informativeness and compactness while excluding redundant nodes and edges that do not contribute to normality.

B. NGLAD: The Subgraph Extractor

For an input graph G , the Subgraph Extractor $\Psi(G; \Theta)$ firstly learns the node representations $\{\mathbf{h}(u_i) \in \mathbb{R}^q | u_i \in \mathcal{V}_G\}$ through a GNN backbone. A multi-layer perceptron (MLP) with a sigmoid activation is then applied to these node representations to compute the probability vector $\mathbf{p} \in \mathbb{R}^n$, where each entry $p(u_i)$ denotes the probability of sampling node u_i to form the subgraph $G^{(S)}$, as shown below:

$$p(u_i) = \sigma(\text{MLP}(\mathbf{h}(u_i))). \quad (4)$$

Nodes are then sampled according to these probabilities, i.e., $a_{u_i} \sim \text{Bernoulli}(p(u_i))$. If $a_{u_i} = 1$, node u_i is selected for the subgraph $G^{(S)}$; otherwise, it is excluded.

However, since the sampling operation makes the gradient with respect to $p(u_i)$ non-computable, it prevents backpropagation and end-to-end training. To overcome this, we use the Gumbel-Softmax reparameterization trick [52]. Instead of directly sampling binary values, a continuous approximation is introduced, where each node in G is assigned a soft weight $\hat{a}_{u_i} \in (0, 1)$, calculated as:

$$\hat{a}_{u_i} = \sigma\left(\frac{\log(\epsilon) - \log(1 - \epsilon) + p(u_i)}{\tau}\right), \quad (5)$$

where $\epsilon \sim \text{Uniform}(0, 1)$ and τ is the temperature parameter. When $\tau \rightarrow 0$, $\hat{a}_{u_i}$ approaches a binary value, effectively approximating the sampling process.

Once the soft node selections are determined, the edge weights for the subgraph $G^{(S)}$ are computed based on these node weights. Specifically, if an edge e_{ij} exists between nodes u_i and u_j in G , the corresponding edge in $G^{(S)}$ is assigned a weight as follows:

$$\hat{e}_{ij} = \hat{a}_{u_i} \cdot \hat{a}_{u_j}. \quad (6)$$

This approach allows the construction of the subgraph $G^{(S)}$ while ensuring differentiability throughout the entire process.

C. NGLAD: The Normality Learner

To guide the Subgraph Extractor to extract the potential normality-relevant subgraphs, the Normality Learner module has two primary functions: (1) assessing the similarity of subgraphs extracted from different training graphs, as normality-relevant subgraphs are expected to capture the shared structural patterns inherent in normal graphs, and (2) ensuring that the representation of $G^{(S)}$ is closely aligned with that of G , as $G^{(S)}$ should effectively capture the most salient features of G , particularly those indicative of its normality. After extracting the subgraph $G^{(S)}$ through the Subgraph Extractor, we employ the Shared GNN Encoder f_{GNN} to represent $G^{(S)}$ and G as $\mathbf{h}(G^{(S)}) \in \mathbb{R}^p$ and $\mathbf{h}(G) \in \mathbb{R}^p$, respectively. The formulations are shown as below:

$$\mathbf{h}(G^{(S)}) = f_{\text{GNN}}(G^{(S)}), \quad (7)$$

and

$$\mathbf{h}(G) = f_{\text{GNN}}(G). \quad (8)$$

Next, we will detail how the Normality Learner evaluates subgraph similarity and measures the alignment in representations.

1) *Establishing Subgraph Similarity*: We assume that the representation process accomplished by the Shared GNN Encoder incurs no information loss, thereby allowing us to establish subgraph similarity within the graph representation space. To prompt the similarity among subgraphs $G^{(S)}$ extracted from various input graphs, we aim to make the representations of subgraph extracting from different inputs conform to the Gaussian distribution $P_{\text{normal}} = \mathcal{N}(\mathbf{r}, \mathbf{\Sigma})$, where $\mathbf{r} \in \mathbb{R}^p$ is the mean vector, $\mathbf{\Sigma} \in \mathbb{R}^{p \times p}$ is the covariance matrix.

We firstly utilize a Gaussian Encoder f_{Gau} to standardize all subgraph representations $\mathbf{h}(G^{(S)})$ output by the Shared GNN encoder, shown as below:

$$\mathcal{P}(\hat{\mathbf{h}}(G^{(S)})|\mathbf{h}(G^{(S)})) = \mathcal{N}(\mathbf{u}(G^{(S)}), \sigma(G^{(S)})\mathbf{I}), \quad (9)$$

where

$$\mathbf{u}(G^{(S)}) = f_{\text{Gau}}^{\mu}(\mathbf{h}(G^{(S)})) \quad (10)$$

and

$$\sigma(G^{(S)}) = f_{\text{Gau}}^{\sigma}(\mathbf{h}(G^{(S)})), \quad (11)$$

where $\mathbf{u}(G^{(S)}) \in \mathbb{R}^p$ represents the mean vector, $\sigma(G^{(S)})\mathbf{I}$ represents the covariance matrix, $\sigma(G^{(S)}) \in \mathbb{R}^p$ and $\mathbf{I} \in \mathbb{R}^{p \times p}$ is an identity matrix. To obtain $\hat{\mathbf{h}}(G^{(S)}) \sim \mathcal{N}(\mathbf{u}(G^{(S)}), \sigma(G^{(S)})\mathbf{I})$,

we employ the reparameterization trick [53] for gradient estimation, expressed as follows:

$$\hat{\mathbf{h}}(G^{(S)}) = \mathbf{u}(G^{(S)}) + \sigma(G^{(S)})\mathbf{I} \odot \epsilon, \quad (12)$$

where $\epsilon \in \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the independent Gaussian noise and $\odot$ denotes the element-wise product.

Further, the similarity of subgraphs $G^{(S)}$ extracted from different input graphs G will be accomplished by minimizing the Kullback leibler (KL) divergence [54] between $\mathcal{N}(\mathbf{u}(G^{(S)}), \sigma(G^{(S)})\mathbf{I})$ and P_{normal} . The KL divergence can be calculated as follows:

$$\begin{aligned} \mathcal{L}_1 &= \min_{\Theta, \Phi} \frac{1}{N} \sum_{G^{(S)}} \text{KL}[\mathcal{N}(\mathbf{u}(G^{(S)}), \sigma(G^{(S)})\mathbf{I}) \parallel \mathcal{N}(\mathbf{r}, \sigma^2\mathbf{I})] \\ &= \min_{\Theta, \Phi} \frac{1}{Np} \sum_{G^{(S)}} \sum_{t=1}^p \log \frac{\sigma}{\sigma(G^{(S)})_t} \\ &\quad + \frac{\sigma(G^{(S)})_t^2 + (\mu(G^{(S)})_t - \mathbf{r}_t)^2}{2\sigma^2} - \frac{1}{2}, \end{aligned} \quad (13)$$

For $P_{\text{normal}} = \mathcal{N}(\mathbf{r}, \mathbf{\Sigma})$, we derive $\mathbf{r}$ by averaging all subgraph representations $\hat{\mathbf{h}}(G^{(S)})$ and we set $\mathbf{\Sigma} = \sigma^2\mathbf{I}$, where σ is a hyperparameter. During training, the mean vector $\mathbf{r}$ of P_{normal} will be updated every m epochs through:

$$\mathbf{r} \leftarrow \frac{1}{N} \sum_{G \in \mathcal{G}} \hat{\mathbf{h}}(G^{(S)}). \quad (14)$$

2) *Measuring Representation Alignment*: In the normality learner, following the Shared GNN Encoder, apart from the Gaussian Encoder, we also have a Fixed Random MLP with its parameters fixed to the random initialization. This path is specialized in representing graph G , with the representation defined as:

$$\hat{\mathbf{h}}(G) = f_{\text{MLP}}(\mathbf{h}(G)). \quad (15)$$

To make the representation of $G^{(S)}$ is closely aligned with that of G , we minimize the difference between $\hat{\mathbf{h}}(G^{(S)})$ and $\hat{\mathbf{h}}(G)$:

$$\mathcal{L}_2 = \min_{\Theta, \Phi} \frac{1}{N} \sum_{G \in \mathcal{G}} \|\hat{\mathbf{h}}(G) - \hat{\mathbf{h}}(G^{(S)})\|^2. \quad (16)$$

By randomly initializing the parameters of f_{MLP} and keeping them fixed during training, it effectively acts as a fixed random projection that can only output stable features from the input graph G . This design requires the subgraph extractor to select subgraphs $G^{(S)}$ that can capture these stable normality features from G . Moreover, gradient backpropagation from f_{MLP} to the shared GNN encoder is blocked, preventing interference with the learning of graph representations. As a result, the Subgraph Extractor bears the primary responsibility for capturing essential information, rather than allowing the Shared GNN Encoder to compensate for suboptimal subgraph selection.

D. Model Training

The distortion term that evaluates the discrepancies between the extracted subgraphs and the inherent normality can be formulated as:

$$\mathcal{D}(\Psi(G; \Theta), \Phi; G) = \mathcal{L}_1 + \mathcal{L}_2. \quad (17)$$

Minimizing this term encourages the Subgraph Extractor $\Psi(G; \Theta)$ to identify normality-relevant subgraphs from each input graph G . However, optimizing this term alone cannot guarantee that $G^{(S)}$ excludes nodes and edges unrelated to the normality. To address this, we also minimize the mutual information term $I(\Psi(G; \Theta); G)$, i.e., $I(G; G^{(S)})$, while quantifies the information retained $G^{(S)}$ about G . Since directly minimizing $I(G; G^{(S)})$ is intractable, we instead focus on finding its upper bound, with the derivation process below:

$$\begin{aligned}
I(G^{(S)}; G) &= \sum_{G^{(S)}, G} P(G^{(S)} | G) P(G) \log \frac{P(G^{(S)} | G)}{P(G^{(S)})} \\
&= \sum_{G^{(S)}, G} P(G^{(S)} | G) P(G) \log \frac{P(G^{(S)} | G)}{P(G^{(S)})} \cdot \frac{Q(G^{(S)})}{Q(G^{(S)})} \\
&= \sum_{G^{(S)}, G} P(G^{(S)} | G) P(G) \log \frac{P(G^{(S)} | G) \cdot Q(G^{(S)})}{Q(G^{(S)}) \cdot P(G^{(S)})} \\
&= \sum_{G^{(S)}, G} P(G^{(S)} | G) P(G) \log \frac{P(G^{(S)} | G)}{Q(G^{(S)})} \\
&\quad - \sum_{G^{(S)}} P(G^{(S)}) \log \frac{P(G^{(S)})}{Q(G^{(S)})} \\
&= \sum_G P(G) \text{KL}[P(G^{(S)} | G) \| Q(G^{(S)})] \\
&\quad - \text{KL}[P(G^{(S)}) \| Q(G^{(S)})] \\
&\leq \sum_G P(G) \text{KL}[P(G^{(S)} | G) \| Q(G^{(S)})], \tag{18}
\end{aligned}$$

where $Q(G^{(S)})$ is the variational approximation of $P(G^{(S)})$. Therefore, minimizing $I(G^{(S)}; G)$ can be transferred to minimize $\sum_G P(G) \text{KL}[P(G^{(S)} | G) \| Q(G^{(S)})]$. As aforementioned in the Sec. IV-B, $P(G^{(S)} | G) = \prod_{i=1}^n p(a_{u_i} | p(u_i))$, $a_{u_i} \sim \text{Bernoulli}(p(u_i))$. Similarity, $Q(G^{(S)} | G) = \prod_{i=1}^n p(b_{u_i} | r)$, $b_{u_i} \sim \text{Bernoulli}(r)$, $r \in (0, 1)$ is a prior parameter. Finally, we can obtain:

$$\begin{aligned}
\mathcal{L}_3 &= \min_{\Theta} \frac{1}{N} \sum_G I(G^{(S)}; G) \\
&= \min_{\Theta} \frac{1}{N} \sum_G \text{KL}[P(G^{(S)} | G) \| Q(G^{(S)})] \\
&= \min_{\Theta} \frac{1}{Nn} \sum_G \sum_{u_i \in \mathcal{V}_G} p(u_i) \log \frac{p(u_i)}{r} \\
&\quad + (1 - p(u_i)) \log \frac{1 - p(u_i)}{1 - r}. \tag{19}
\end{aligned}$$

Thus, during model training, the final optimization objective $\mathcal{L}$ is the summation of $\mathcal{L}_1$, $\mathcal{L}_2$, and $\mathcal{L}_3$:

$$\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2 + \mathcal{L}_3. \tag{20}$$

E. Model Inference

In this part, we will detail how our proposed GLAD model, NGLAD, identifies anomalous graphs and provides explanations for its predictions during the model inference phase.

1) *Identify Anomalous Graphs*: For a test graph G , its anomaly score is defined by the prediction error between the graph and its extracted subgraph representations:

$$\text{score}(G) = \|\hat{\mathbf{h}}(G) - \hat{\mathbf{h}}(G^{(S)})\|^2. \tag{21}$$

During training, NGLAD learns to extract subgraphs $G^{(S)}$ that represent the intrinsic normality of normal graphs. Therefore, during inference, a normal graph G will yield a lower anomaly score. In contrast, NGLAD has not been trained to extract representative subgraph $G^{(S)}$ for anomalous graphs, which results in higher anomaly scores for these instances. This distinction in anomaly scores between normal and anomalous graphs allows NGLAD to effectively identify anomalies.

2) *Explanation*: NGLAD provides explanations by identifying “normality-relevant” subgraphs, which are shared substructures commonly found in normal graphs. For normal graphs, these subgraphs serve as their most discriminative components, explaining why a graph is identified as normal. In practice, altered in normality-relevant subgraphs often signal the presence of anomalies. Specifically, the presence of subgraphs that resemble normal patterns but exhibit critical deviations within a graph indicates a departure from normality, contributing to the anomaly. NGLAD’s ability to identify subgraphs relevant to normality enables it to detect changes in these subgraphs. Thus, NGLAD not only identifies anomalies but also explains the source of abnormality, enhancing the interpretability of its predictions. As aforementioned, $\hat{a}_{u_i}$ and $\hat{e}_{ij}$ represent the soft weights of node u_i and edge e_{ij} in G , respectively. A threshold-based or top-k selection strategy can be applied to obtain the explanation subgraph $\hat{G}^{(S)}$.

F. Model Complexity Analysis

Within this subsection, we denote the number of graphs as N , and the average numbers of nodes and edges as n and m , respectively. At each training epoch, the Subgraph Extractor requires $\mathcal{O}(N(m + n))$ complexity. The GNN consumes $\mathcal{O}(N(Lmd + nd))$, where L denotes the number of graph convolution layers and d represents the latent dimension of graph representations. The distortion terms $\mathcal{L}_1 + \mathcal{L}_2$ require $\mathcal{O}(Nd)$, while the mutual information term $\mathcal{L}_3$ incurs a complexity of $\mathcal{O}(Nn)$. Therefore, the overall time complexity of NGLAD is $\mathcal{O}(N(Lmd + nd + m + n))$. During inference, NGLAD only performs the forward pass through the Subgraph Extractor and GNN module, resulting in a complexity of $\mathcal{O}(N(Lmd + nd + m + n))$ of N graphs. This is the same as the forward computation in training, without the additional overhead of backpropagation and parameter updates. The pseudo code is illustrated in Algorithm 1.

Among recent methods specifically designed for interpretable GLAD, SIGNET [16] has a theoretical time complexity of $\mathcal{O}(N(Lmd + nd + m + n))$, which can be comparable to that of NGLAD. However, this equivalence holds only under strict assumptions: the GNN encoder for the original graph and the HGNN encoder for the hypergraph view must be executed in parallel, and the additional cost of constructing the hypergraph is not considered. In practice, these conditions may not always be achievable, making SIGNET potentially less efficient than

Algorithm 1 Forward Propagation of NGLAD

Input: The training graph set $\mathcal{G}$ and hyper-parameters
Output: Model parameters

- 1 Randomly initialize the model parameters for the Subgraph Extractor Ψ and the Normality Learner Φ , and initialize the mean vector $\mathbf{r}$ of P_{normal} .
- /* Training Stage */
- 2 **for** $epoch = \{1, \dots, E\}$ **do**
- 3 **for** $G \in \mathcal{G}$ **do**
- 4 Extract subgraphs from G via the Subgraph Extractor: $G^{(S)} \leftarrow \Psi(G; \Theta)$ (Eq. (4), (5), (6));
- 5 Represent $G^{(S)}$ as $\mathbf{h}(G^{(S)})$ via Eq. (7);
- 6 Represent G as $\mathbf{h}(G)$ via Eq. (8);
- 7 Obtain $\mu(G^{(S)})$ and $\sigma(G^{(S)})$ via Eq. (10) and Eq. (11), respectively;
- 8 Obtain $\hat{\mathbf{h}}(G^{(S)})$ via Eq. (12);
- 9 Obtain $\hat{\mathbf{h}}(G)$ via Eq. 15
- 10 Calculate the loss objective Eq. (20) via Eq. (13), (16), (19) ;
- 11 Backpropagation and update the parameters;
- 12 **if** $epoch \% m == 0$ **then**
- 13 update $\mathbf{r}$ via Eq. 14).
- /* Inference Stage */
- 14 **for** $G \in \mathcal{G}_{\text{test}}$ **do**
- 15 Calculate graph anomaly score $score(G_i)$ via Eq. 16;
- 16 Obtain the explanation $\hat{G}^{(S)}$.

NGLAD. GLADPro [17] requires $\mathcal{O}(N(Lmd+nd+m+n+N))$, where the extra $\mathcal{O}(N^2)$ complexity is incurred due to the prototype-learning procedure.

V. EXPERIMENTS

In this section, we conduct extensive experiments to answer the following research questions.

- **Q1** Can NGLAD achieve superior performance in quantitative evaluations of explanation performance?
- **Q2** How do the explanations learned by NGLAD perform in qualitative visual evaluations?
- **Q3** Is NGLAD effective in accurately identifying anomalous graphs?
- **Q4** How does each core component of NGLAD contribute to its overall performance?
- **Q5** How does the parameter analysis reveal the impact of various configurations on NGLAD’s performance?
- **Q6** How does the running efficiency of NGLAD compare to other state-of-the-art methods?

A. Experimental Setup

1) *Datasets*: We employ five graph datasets for interpretable GLAD tasks, including two real-world datasets (Mutagen and MNIST) and three synthetic datasets (BA-TYPE, BA-COUNT, and BA-SIZE). All these datasets provide ground-truth explanatory subgraphs, which enable quantitative evaluation of the interpretability of GLAD models.

TABLE I

STATISTICAL INFORMATION OF REAL-WORLD AND SYNTHETIC GRAPH DATASETS. THE LAST COLUMN INDICATES WHETHER EACH DATASET PROVIDES GROUND-TRUTH EXPLANATIONS FOR QUANTITATIVE EVALUATION

Data	# Graphs	Avg. #Nodes	Avg. #Edges	Explanation Eval.
Mutagen	4337	28.25	58.28	✓
MNIST	1500	71.32	599.09	✓
BA-TYPE	1000	40.00	42.62	✓
BA-COUNT	1000	40.00	42.09	✓
BA-SIZE	1000	40.00	40.00	✓
MUTAG	188	19.49	43.55	×
PROTEIN	1113	46.74	176.63	×
DD	1178	333.04	1689.29	×
IMDB-BINARY	1000	19.81	188.69	×
REDDIT-BINARY	2000	429.63	497.75	×
COLLAB	5000	74.49	2457.78	×

- *Mutagen* is a dataset aimed at predicting molecular properties and includes both mutagenic and non-mutagenic compounds [55]. The presence of functional groups such as $-\text{NO}_2$ and $-\text{NH}_2$ typically indicates mutagenicity. In this context, we regard mutagenic molecules as the normal class and consider non-mutagenic molecules as anomalies. The $-\text{NO}_2$ and $-\text{NH}_2$ groups in the mutagenic compounds serve as the normality-relevant subgraphs.
- *MNIST* is derived from the MNIST-75sp superpixel dataset [56] and has been converted into graph-structured data. We randomly select three out of the ten classes, corresponding to the digits 0 to 9, to represent the normal, while the remaining classes are identified as anomalies. In the normal graphs, the segments most closely associated with the represented digit are identified as normality-relevant subgraphs.
- *BA-TYPE*, *BA-COUNT*, and *BA-SIZE* are synthetic datasets created using widely synthetic benchmarks [13], utilizing Barabási-Albert (BA) graphs that incorporate various motifs. In the BA-TYPE dataset, each normal graph contains one of the following normality-relevant subgraphs: “house”, “wheel”, or “lattice”, while anomalies are characterized by either a “diamond” or a “5-cycle”. For the BA-COUNT dataset, normal graphs include 1 or 2 “house” as normality-relevant subgraphs, whereas anomalies contain 3 or 4 “house”. In the BA-SIZE dataset, normal graphs feature “cycle” with 3 to 5 nodes as normality-relevant subgraphs, while anomalies consist of “cycle” with 6 to 8 nodes.

Additionally, to assess the model performance in detecting anomalous graphs, we utilize six supplementary datasets from the TU collection [57]. Comprehensive statistical details regarding these datasets can be found in Table I.

2) *Comparison Methods*: For evaluating our proposed method in identifying anomalous graphs, we employ six deep GLAD algorithms for comparison:

- OCGIN [7] combines one-class classification with GNNs to detect anomalous graphs. This method aims to map normal graphs onto a hypersphere and detect anomalies by measuring their deviation from it.
- OCGTL [6] integrates deep one-class classification with self-supervision. The approach involves $k + 1$ GNNs trained on two complementary losses.

- GLocalKD [8] trains a predictor GNN to replicate random GNN representations, forcing it to learn major information in the normal graphs.
- GOOD-D [51] employs hierarchical contrastive learning to distinguish normal graphs at the node, graph, and group levels. Then, it identifies anomalies by measuring the disagreement between different graph views and their alignment with normal graphs.
- SIGNET [16] evaluates graph abnormality by learning the cross-view mutual information between the original graph and its hypergraph representation.
- GLADPro [17] is an interpretable GLAD framework based on prototype learning. It models the distribution of normal graphs using a set of vectorized prototypes. Two variants are proposed: GLADPro-Instance, which generates explanations independently for each graph, and GLADPro-Global, which learns prototype representations to interpret GLAD and heuristically maps them to concrete subgraphs.

Among the methods introduced above, only SIGNET and GLADPro are capable of addressing the interpret GLAD problem. To obtain GLAD explanations for the remaining four methods, we incorporate three post-hoc GNN explainers:

- GNNExplainer [58] (short for GNNEx) identifies compact subgraphs and key node features that are most influential for a GNN's prediction. It can handle both single- and multi-instance explanations, offering interpretable insights into the model's decisions.
- PGExplainer [34] (short for PGEx) provides global and inductive explanations for GNNs by learning generative models of underlying graph structures. Shared neural network parameters allow it to collectively explain multiple instances and generalize to unseen nodes efficiently.
- GraphMaskExplainer [59] (short for MaskEx) interprets GNNs via a differentiable erasure function that decides which edges to retain or discard for faithful explanations. By amortizing learning over the dataset, it avoids hindsight bias while remaining scalable and model-agnostic.

The GLAD baseline models, including OCGIN, OCGTL, GLocalKD, and GOOD-D, follow a “detector + explainer” framework and are trained solely on normal graphs. In this framework, the detector assigns an anomaly score to each graph, while the explainer highlights subgraphs that are most responsible for lowering the anomaly score.

3) *Metrics*: To evaluate GLAD performance, we calculate the ROC-AUC based on anomaly scores and their corresponding normal/anomalous labels, which we refer to as AD-AUC. For the interpretation assessment, we provide explanation ROC-AUCs at both the node level (N-AUC) and the edge level (E-AUC). Specifically, nodes and edges within the ground-truth explanation subgraph are labeled as 1, while all others are assigned a label of 0. In NGLAD, the soft weights $\hat{a}_{u_i}$ and $\hat{e}_{ij}$ generated by the Subgraph Extractor serve as prediction logits. This allows us to compute AUCs between these prediction logits and the ground-truth as performance metrics.

We conduct each experiment five times with different random seeds and report the mean and standard deviation of the evaluation metrics.

4) *Implementation*: For NGLAD, we employ GIN [26] as the GNN backbone to derive node and graph representations. We conduct a grid search to identify the key hyperparameters for both NGLAD and the baseline methods. All experiments are implemented on Python 3.9.0, CUDA 11.6.1, a Rocky Linux 8.6 (Green Obsidian) server with a 12-core CPU, 1 NVIDIA V100 GPU, PyTorch 1.10.0, and 300GB RAM.

B. Quantitative Evaluation of Explainability (Q1)

As shown in Table II, We evaluate model performance using N-AUC and E-AUC on five graph datasets with ground-truth explanations. These metrics quantitatively assess explainability by measuring the model's ability to correctly identify nodes (N-AUC) and edges (E-AUC) within the ground-truth explanatory subgraphs. Among the post-hoc explainers, PGEx+ provides only edge-level explanations, whereas MaskEx+ and GNNEx+ can generate both node- and edge-level explanations.

According to the reported model performance, we observe the following key findings: (1) NGLAD demonstrates superior performance in interpreting GLAD predictions. On datasets including MNIST, BA-TYPE, BA-COUNT, and BA-SIZE, NGLAD consistently outperforms comparison methods in both node-level and edge-level explanations, demonstrating its overall superiority. Specifically, it achieves an average improvement of 2.34 % in N-AUC and 1.72% in E-AUC over the best-performing baseline GLADPro-Global. On the Mutagen dataset, NGLAD attains 99.42% and 99.65% regarding N-AUC and E-AUC, respectively. Although GLADPro-Global can reach 100% in N-AUC, it requires predefining the number of prototypes as two, since the normal class in Mutagen contains two normality-relevant subgraphs, i.e., NH_2 and NO_2 . When the number of normality-relevant subgraphs is unknown, GLADPro-Instance experiences a drop in N-AUC from 100% to 95.83%; (2) Built-in interpretability delivers more effective explanations than post-hoc approaches. Models with built-in interpreters, such as SIGNET, GLADPro and NGLAD, are more effective at capturing key distinguishing features between normal and anomalous graphs on real-world datasets. In contrast, post-hoc approaches show substantial performance fluctuations across these datasets, highlighting their limitations in adapting to varying data conditions. On synthetic datasets, however, SIGNET demonstrates suboptimal performance. This instability may stem from the lack of labels providing reliable supervision for explanations.

C. Qualitative Evaluation of Explainability (Q2)

To further demonstrate NGLAD's explanatory power, we show the subgraphs extracted by interpretable GLAD models on the Mutagen (Fig. 3), BA-TYPE (Fig. 4), and IMDB-BINARY (Fig. 5) datasets.

As shown in Fig. 3, we present the explanation results for five randomly sampled mutagenic molecules. Our proposed method, NGLAD, effectively explains these mutagenic molecules without including any irrelevant nodes or edges. In

TABLE II

EXPLAINABILITY PERFORMANCE REGARDING N-AUC AND E-AUC (IN PERCENT, MEAN $\pm$ STD). "PGEX +", "MASKEX +", AND "GNNEX +" MEANS ATTACHING POST-HOC EXPLAINER TO GLAD MODELS BELOW. THE BOLD FONT HIGHLIGHTS THE BEST PERFORMANCE

	Dataset	Mutagen		MNIST		BA-TYPE		BA-COUNT		BA-SIZE	
Post-hoc Explainer	Metric Method	N-AUC	E-AUC	N-AUC	E-AUC	N-AUC	E-AUC	N-AUC	E-AUC	N-AUC	E-AUC
PGEx+	OCGIN	-	76.99 $\pm$ 1.16	-	37.21 $\pm$ 3.89	-	42.25 $\pm$ 0.34	-	49.95 $\pm$ 0.47	-	50.01 $\pm$ 0.46
	OCGTL	-	68.28 $\pm$ 2.49	-	53.23 $\pm$ 3.07	-	48.98 $\pm$ 4.61	-	65.03 $\pm$ 0.71	-	54.53 $\pm$ 1.26
	GLocalKD	-	80.01 $\pm$ 3.45	-	49.11 $\pm$ 5.22	-	50.98 $\pm$ 3.76	-	69.23 $\pm$ 0.76	-	60.11 $\pm$ 3.20
	GOOD-D	-	92.07 $\pm$ 4.36	-	45.01 $\pm$ 5.47	-	52.23 $\pm$ 5.86	-	66.59 $\pm$ 1.79	-	56.07 $\pm$ 2.27
MaskEx+	OCGIN	49.44 $\pm$ 0.62	45.05 $\pm$ 0.70	49.98 $\pm$ 0.42	46.18 $\pm$ 0.69	50.48 $\pm$ 0.39	51.43 $\pm$ 2.68	50.68 $\pm$ 0.39	49.96 $\pm$ 0.59	50.68 $\pm$ 0.06	49.88 $\pm$ 0.48
	OCGTL	50.09 $\pm$ 0.84	62.08 $\pm$ 4.62	59.48 $\pm$ 0.21	57.73 $\pm$ 4.25	50.55 $\pm$ 0.29	54.19 $\pm$ 1.95	50.51 $\pm$ 0.01	53.23 $\pm$ 1.84	50.51 $\pm$ 0.01	52.65 $\pm$ 1.33
	GLocalKD	53.09 $\pm$ 1.54	60.01 $\pm$ 0.33	62.09 $\pm$ 1.23	59.33 $\pm$ 1.90	57.20 $\pm$ 3.22	59.01 $\pm$ 3.45	53.01 $\pm$ 0.34	54.11 $\pm$ 0.52	52.11 $\pm$ 0.66	50.76 $\pm$ 0.79
	GOOD-D	50.62 $\pm$ 0.99	56.26 $\pm$ 6.40	50.14 $\pm$ 0.18	54.89 $\pm$ 0.71	50.04 $\pm$ 0.20	53.28 $\pm$ 4.08	49.92 $\pm$ 0.71	50.51 $\pm$ 8.41	49.92 $\pm$ 0.71	49.02 $\pm$ 7.55
GNNEx+	OCGIN	47.83 $\pm$ 7.55	47.47 $\pm$ 1.37	58.77 $\pm$ 0.74	50.89 $\pm$ 0.28	73.37 $\pm$ 4.81	61.09 $\pm$ 1.02	68.34 $\pm$ 3.82	61.45 $\pm$ 1.76	68.56 $\pm$ 1.67	62.24 $\pm$ 0.56
	OCGTL	61.85 $\pm$ 1.99	65.22 $\pm$ 2.91	51.87 $\pm$ 1.64	53.69 $\pm$ 3.27	77.75 $\pm$ 0.84	62.20 $\pm$ 1.58	72.31 $\pm$ 1.19	54.73 $\pm$ 2.94	51.45 $\pm$ 2.05	53.00 $\pm$ 1.81
	GLocalKD	56.20 $\pm$ 1.23	55.09 $\pm$ 1.35	53.11 $\pm$ 0.98	52.98 $\pm$ 1.01	73.89 $\pm$ 2.23	65.22 $\pm$ 2.01	71.22 $\pm$ 2.78	60.10 $\pm$ 2.23	59.11 $\pm$ 1.76	55.23 $\pm$ 2.11
	GOOD-D	53.70 $\pm$ 2.01	53.05 $\pm$ 2.31	60.58 $\pm$ 2.57	50.93 $\pm$ 0.66	67.79 $\pm$ 6.36	46.45 $\pm$ 5.25	70.50 $\pm$ 9.81	55.57 $\pm$ 1.30	67.21 $\pm$ 6.06	54.04 $\pm$ 3.66
-	SIGNET	72.59 $\pm$ 13.83	74.08 $\pm$ 14.41	76.15 $\pm$ 3.35	82.25 $\pm$ 3.13	62.22 $\pm$ 9.08	67.34 $\pm$ 8.73	60.36 $\pm$ 7.40	68.61 $\pm$ 9.42	60.97 $\pm$ 6.93	68.47 $\pm$ 9.35
-	GLADPro-Instance	95.83 $\pm$ 8.06	96.69 $\pm$ 5.55	81.70 $\pm$ 3.15	80.32 $\pm$ 3.85	88.27 $\pm$ 0.68	93.35 $\pm$ 0.22	71.66 $\pm$ 3.20	74.09 $\pm$ 3.03	75.76 $\pm$ 8.75	75.09 $\pm$ 6.23
	GLADPro-Global	100.00$\pm$0.00	99.76$\pm$0.48	82.85 $\pm$ 3.33	84.77 $\pm$ 3.82	92.64 $\pm$ 2.39	96.92 $\pm$ 0.27	81.04 $\pm$ 1.56	84.60 $\pm$ 1.68	85.03 $\pm$ 0.95	85.51 $\pm$ 1.67
-	NGLAD	99.42 $\pm$ 0.01	99.65 $\pm$ 0.01	87.33$\pm$2.01	86.32$\pm$1.11	92.77$\pm$1.88	96.98$\pm$1.76	84.32$\pm$2.98	86.09$\pm$1.09	85.22$\pm$2.22	88.28$\pm$1.72

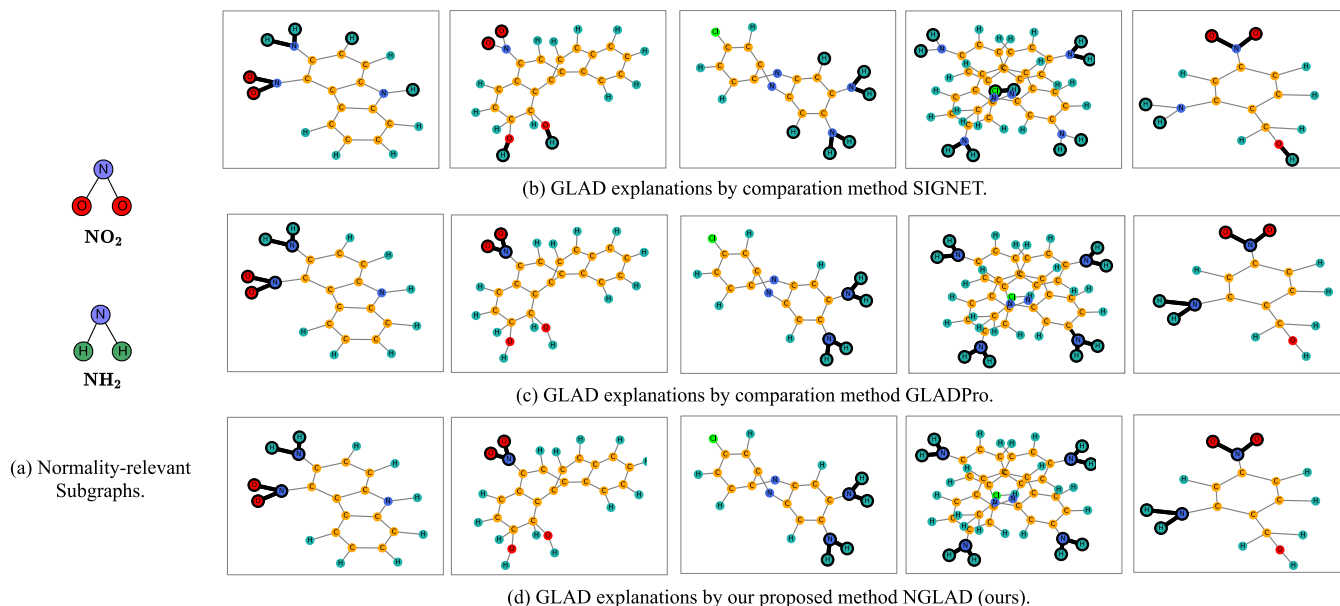


Fig. 3. The visualizations illustrate the explainable results for normal graphs from the real-world dataset Mutagen. Notably, functional groups such as NH_2 and NO_2 are emphasized as normality-relevant subgraphs, as their presence typically indicates mutagenicity. In contrast, anomalous, non-mutagenic molecules lack these functional groups. (a) focuses on the normality-relevant subgraphs NH_2 and NO_2 . (b) presents the results from the state-of-the-art method, SIGNET. (c) shows the results from the state-of-the-art method GLADPro. (d) displays the explainable results obtained using our proposed method, NGLAD. Bold black edges and nodes represent the top- k ranked elements based on their soft weights, with k maintained at the same value for different methods.

other words, NGLAD classifies these molecules as normal by identifying the normality-relevant subgraphs NO_2 and NH_2 . In contrast, the comparison method SIGNET fails to provide complete explanations for these normal graphs, as it does not

include the crucial nitrogen (N) atom in these randomly sampled graphs. Additionally, it includes some redundant nodes and edges in its explanations, suggesting that SIGNET may generate inaccurate explanation results. In addition, although

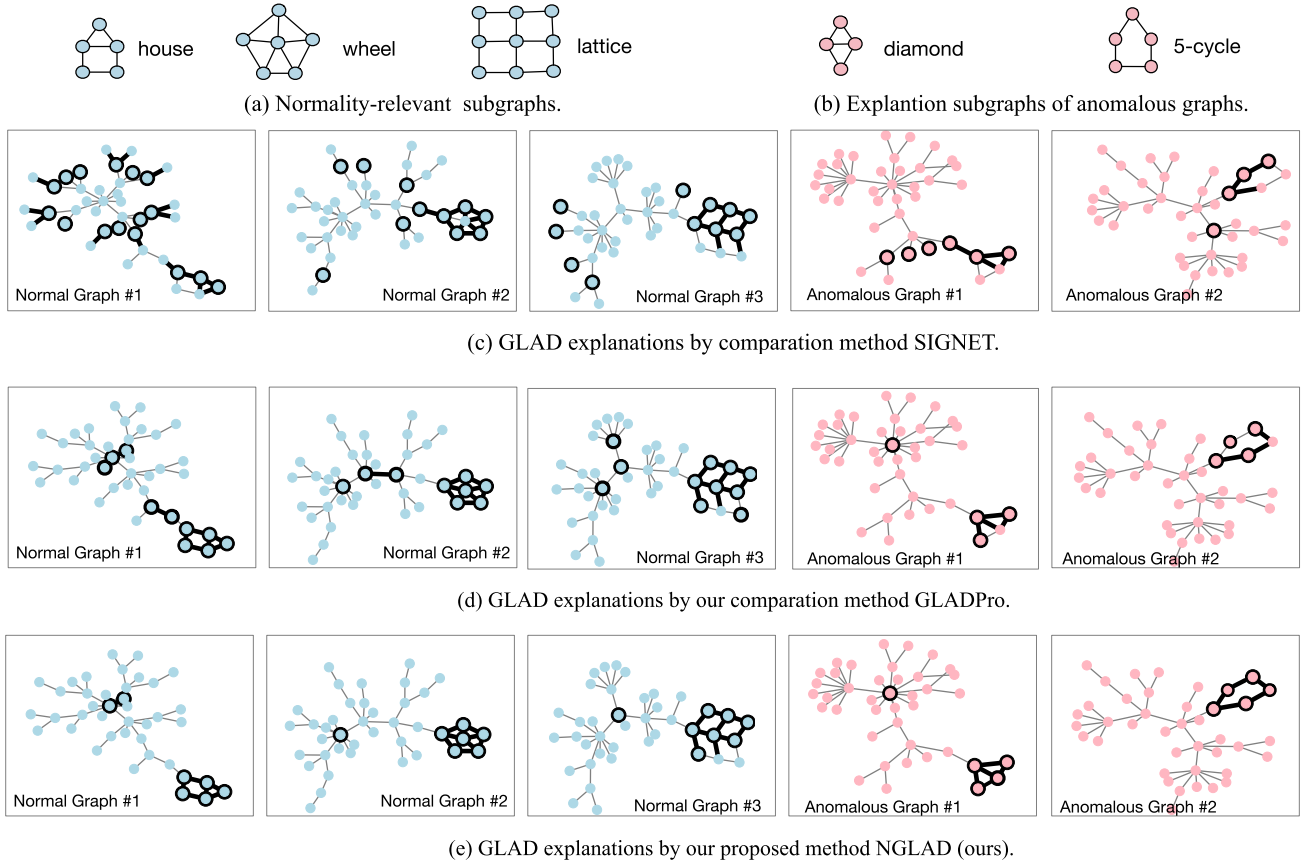


Fig. 4. The visualizations illustrate the explainable results for both normal and anomalous graphs from the synthetic dataset BA-TYPE. In (a), three normality-relevant subgraphs, i.e., “house”, “wheel”, and “lattice” are highlighted to explain normal graphs. (b) focuses on two explanation subgraphs associated with anomalous graphs, i.e., “diamond” and “5-cycle”. (c) presents the results from the state-of-the-art method, SIGNET. (d) shows the results from the state-of-the-art method GLADPro. (e) displays the explainable results obtained using our proposed method, NGLAD. Bold black edges and nodes represent the top- k ranked elements based on their soft weights, with k consistently set for different methods.

GLADPro generally provides more accurate explanations, in the most complex graph where the ground-truth explanation includes four NH_2 subgraphs, the presence of an irrelevant edge indicates that the prototype may overgeneralize and reveals its limitations in capturing intricate local structures.

As shown in Fig. 4, compared to SIGNET and GLADPro, NGLAD delivers more accurate explanations for normal and anomalous graphs from BA-TYPE. The subgraph explanations for normal graphs produced by SIGNET and GLADPro often include more nodes and edges that are not relevant to the graphs’ normal properties. This tendency makes the explanation results difficult to interpret. In contrast, the explanations produced by NGLAD are more informative and compact. Notably, NGLAD effectively identifies the explanation subgraphs for anomalous graphs. When the explanatory subgraphs of anomalous graphs closely resemble the subgraphs relevant to normality, such as the anomalous explanatory subgraphs “diamond” and “5-cycle”, which are components of the normality-relevant subgraphs “wheel” and “house”, respectively. The experimental results support our opinion that NGLAD’s ability to identify normality-relevant subgraphs enables it to detect disruptions in these subgraphs.

As shown in Fig. 5, NGLAD effectively extracts normality-relevant subgraphs from the IMDB-BINARY dataset. In

multi-group networks (i.e., social networks composed of two or more groups), it identifies connections between central nodes and nodes in the largest group as normality-relevant, consistent with the observation that central nodes and dominant groups typically govern social networks. In contrast, for ring-shaped networks without central nodes, NGLAD considers all nodes and edges as normality-relevant, reflecting that each node contributes equally to maintaining connectivity.

D. Graph-Level Anomaly Detection Results (Q3)

To evaluate NGLAD’s performance in detecting anomalous graphs, we conducted experiments on eleven graph datasets, including both real-world and synthetic datasets. The results are presented in Table III.

On real world dataset, our observations are as follows: (1) NGLAD ranks as the leading method. It achieves the highest AD-AUC scores across all real-world datasets, including Mutagen ($94.35\% \pm 1.23\%$), MNIST ($66.10\% \pm 0.91\%$), MUTAG ($93.08\% \pm 0.11\%$), PROTEIN ($79.87\% \pm 1.43\%$), DD($81.29\% \pm 1.37\%$), IMDB-BINARY ($69.89\% \pm 1.99\%$), REDDIT-BINARY ($89.01\% \pm 0.48\%$), and COLLAB ($74.77\% \pm 1.30\%$). With an average ranking of 1.4,

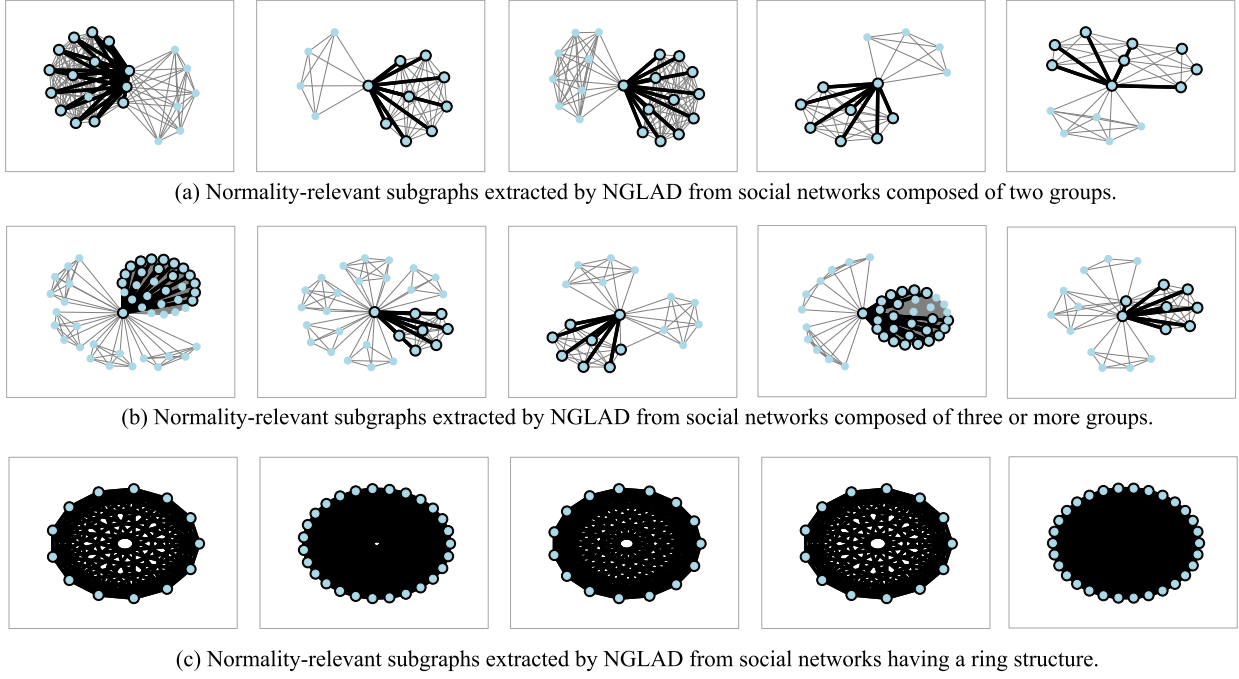


Fig. 5. The visualizations show normality-relevant subgraphs extracted by our proposed method NGLAD from the IMDB-BINARY dataset.

TABLE III
GLAD PERFORMANCE REGARDING AD-AUC (IN PERCENT, MEAN $\pm$ STD). THE BOLD FONT HIGHLIGHTS THE BEST RESULT

Dataset Method	Mutagen	MNIST	BA-TYPE	BA-COUNT	BA-SIZE	MUTAG	PROTEIN	DD	IMDB-BINARY	REDDIT-BINARY	COLLAB	Avg.Rank
OCGIN	88.04 $\pm$ 0.91	61.87 $\pm$ 4.91	85.37 $\pm$ 2.63	95.02 $\pm$ 0.18	95.89 $\pm$ 0.28	59.75 $\pm$ 6.36	72.02 $\pm$ 1.79	71.04 $\pm$ 2.06	53.24 $\pm$ 3.11	72.88 $\pm$ 3.22	70.94 $\pm$ 2.66	5.2
OCGTL	89.51 $\pm$ 0.75	59.93 $\pm$ 4.35	93.93$\pm$1.47	97.22$\pm$0.20	96.30 $\pm$ 0.24	84.09 $\pm$ 3.07	75.12 $\pm$ 1.86	76.64 $\pm$ 5.16	58.26 $\pm$ 4.53	76.11 $\pm$ 1.02	69.70 $\pm$ 0.87	3.5
GLocalKD	71.58 $\pm$ 1.81	56.26 $\pm$ 3.68	78.62 $\pm$ 4.71	80.29 $\pm$ 3.11	69.23 $\pm$ 2.67	76.43 $\pm$ 5.82	71.25 $\pm$ 3.89	79.51 $\pm$ 1.90	61.72 $\pm$ 4.37	78.27 $\pm$ 2.43	52.03 $\pm$ 1.00	4.6
GOOD-D	84.64 $\pm$ 3.47	51.14 $\pm$ 7.23	81.33 $\pm$ 3.72	48.54 $\pm$ 9.71	43.35 $\pm$ 8.72	70.71 $\pm$ 7.66	73.24 $\pm$ 3.08	72.25 $\pm$ 1.54	64.30 $\pm$ 4.15	85.11 $\pm$ 1.92	71.03 $\pm$ 0.30	5.2
SIGNET	81.49 $\pm$ 8.51	42.76 $\pm$ 9.26	82.90 $\pm$ 9.81	89.33 $\pm$ 8.64	86.08 $\pm$ 7.21	75.82 $\pm$ 6.33	73.60 $\pm$ 2.82	73.63 $\pm$ 1.37	62.24 $\pm$ 9.01	83.02 $\pm$ 1.26	62.28 $\pm$ 4.75	5.2
GLADPro	92.23 $\pm$ 1.43	62.43 $\pm$ 5.64	90.66 $\pm$ 2.46	96.92 $\pm$ 6.16	97.51$\pm$0.79	85.94 $\pm$ 2.30	77.68 $\pm$ 1.59	75.10 $\pm$ 1.62	68.41 $\pm$ 2.06	87.38 $\pm$ 1.23	71.65 $\pm$ 2.54	2.2
NGLAD	94.35$\pm$1.23	66.10$\pm$0.91	90.88 $\pm$ 2.10	95.22 $\pm$ 4.33	96.89 $\pm$ 2.00	93.08$\pm$0.11	79.87$\pm$1.43	81.29$\pm$1.37	69.89$\pm$1.09	89.01$\pm$0.48	74.77$\pm$1.30	1.4

NGLAD demonstrates consistent and robust anomaly detection performance across diverse graph structures; (2) NGLAD balances anomaly detection and explanation. It surpasses SIGNET in both anomaly detection and explanation performance, providing more reliable and interpretable results for the GLAD task. While GLADPro shows competitive explanation performance on Mutagen, its overall detection ability on real-world datasets is weaker than NGLAD, likely because its reliance on the prototype learning limits flexibility in capturing the diverse and complex structures of real anomalous graphs.

On synthetic datasets such as BA-TYPE and BA-COUNT, OCGTL performs well because its one-class classification combined with self-supervised graph transformations effectively captures the dominant, uniform structural patterns of normal graphs with small size. However, OCGTL’s performance declines on complex real-world datasets, where normal graphs exhibit complex distributions. In contrast, NGLAD maintains competitive detection performance across both synthetic and real-world datasets, while providing interpretable

insights into the mechanisms underlying normal and abnormal graph properties. On BA-SIZE, GLADPro slightly outperforms our method in AD-AUC (97.51% vs. 96.89%), likely because the anomalies in this dataset are primarily reflected in subgraph size, which GLADPro captures more directly. NGLAD, in contrast, focuses on extracting interpretable normality-relevant subgraphs, highlighting local structural patterns rather than size differences.

E. Ablation Study (Q4)

We conduct ablation studies from three perspectives: (1) NGLAD w/o MI term. We evaluate the importance of the mutual information term by replacing the Bernoulli sampling procedure with the setting $\hat{u}_{u_i} = p(u_i)$ and removing the mutual information term (i.e., $\mathcal{L}_2$ from the final loss function in Eq (14)). (2) NGLAD w/o Gaussian Encoder. We assess the role of the Gaussian Encoder by substituting it with a MLP and omitting the $\mathcal{L}_1$ term from the final loss function Eq (14). (3) NGLAD w/o Fixed Random MLP. We examine the

TABLE IV
ABLATION STUDY RESULTS HIGHLIGHTING THE KEY COMPONENTS OF NGLAD: THE MUTUAL INFORMATION (MI) TERM, GAUSSIAN ENCODER, AND FIXED RANDOM MLP

Dataset	Mutagen			MNIST			BA-TYPE		
Metric Method	AD-AUC	N-AUC	E-AUC	AD-AUC	N-AUC	E-AUC	AD-AUC	N-AUC	E-AUC
w/o MI term	81.91±2.9	81.73±18.25	90.05±9.16	64.09±3.78	84.80±1.60	83.25±1.95	84.02±6.40	67.02±1.3	71.95±4.19
w/o Gaussian Encoder	82.95±1.80	64.98±11.11	71.05±10.96	64.70±2.80	84.57±1.30	82.21±1.3	63.75±3.6	80.78±1.80	82.95±1.13
w/o Fixed Random MLP	68.39±8.4	81.14±10.69	89.58±9.29	65.10±2.67	85.47±1.70	84.08±1.79	85.04±2.70	73.89±4.9	74.62±11.98
NGLAD	94.35±1.23	99.42±0.01	99.65±0.01	66.10±0.91	87.33±2.01	86.32±1.11	90.88±2.10	92.77±1.88	96.98±1.76

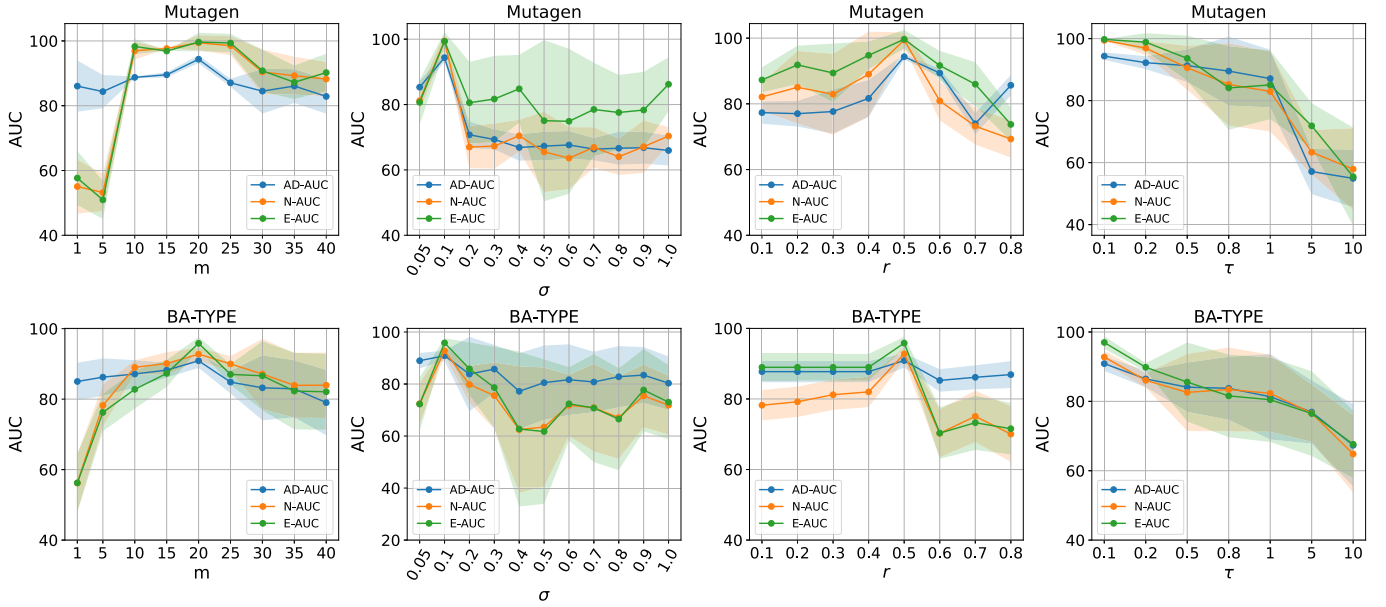


Fig. 6. Parameter analysis of NGLAD with respect to four key hyperparameters: m (update interval for the mean vector of P_{normal}), σ (predefined variance of P_{normal}), r (prior Bernoulli distribution parameter), and τ (Gumbel–Softmax temperature) on the real-world dataset Mutagen and the synthetic dataset BA-TYPE.

significance of the Fixed Random MLP by removing it from the NGLAD framework. The experimental results are presented in Table IV, which indicate that NGLAD consistently achieves the highest AUC scores (AD-AUC, N-AUC, and E-AUC) across multiple datasets, including Mutagen, MNIST, and BA-TYPE. In contrast, obvious performance drop on datasets Mutagen and BA-TYPE can be observed when removing components from NGLAD. This indicates that after losing certain important components, NGLAD becomes more sensitive to variations in the data. Overall, the mutual information term, Gaussian Encoder, and Fixed Random MLP, therefore, contribute positively to NGLAD’s leading performance.

F. Parameter Analysis (Q5)

As illustrated in Fig. 6, we conduct a comprehensive parameter analysis on the real-world dataset Mutagen and the synthetic dataset BA-TYPE, focusing on four key hyperparameters: (1) m — update interval for the mean vector $\mathbf{r}$ of P_{normal} . Our proposed method NGLAD consistently achieves strong performance when m is set to 10, 15, or 20, with the

optimal result at $m = 20$. When m is too small (e.g., 1 or 5), frequent updates cause P_{normal} to fluctuate, while overly large m values make updates too sparse to capture distributional shifts, both leading to performance degradation. (2) σ — predefined variance of P_{normal} . We observe that $\sigma = 0.1$ yields the best performance on both datasets. This indicates that normal graphs are relatively concentrated due to the presence of normality-relevant subgraphs, and a smaller σ allows NGLAD to better fit the underlying data distribution. (3) r — prior Bernoulli distribution parameter. We define $b_{u_i} \sim \text{Bernoulli}(r)$ with $r \in (0, 1)$. In our experiments, r is initialized to 0.9 and decays by 0.1 every 10 epochs until reaching a predefined minimum value. By varying this minimum from 0.8 to 0.1, we find that NGLAD achieves optimal performance at $r = 0.5$, where the variational distribution Q assigns each node an equal prior probability of being sampled. This encourages the model to learn from a diverse set of subgraph instances. (4) τ — Gumbel–Softmax temperature parameter. The temperature τ in the Gumbel–Softmax reparameterization controls how closely the soft weight $\hat{a}_{u_i}$ approximate binary values.

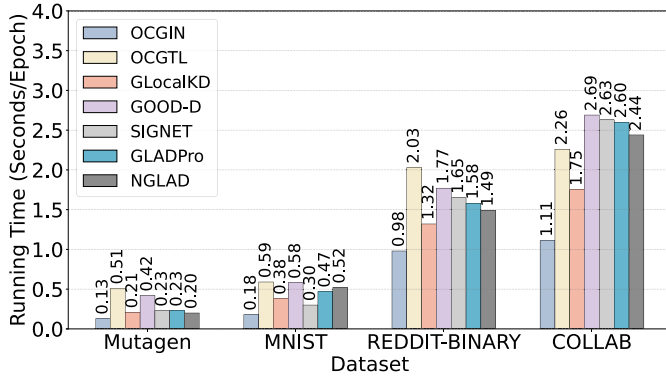


Fig. 7. Comparison of average training time per epoch (in seconds) on four real-world datasets: Mutagen, MNIST, REDDIT-BINARY, and COLLAB.

We find that $\tau = 0.1$ yields the best performance, while larger values gradually degrade it. This aligns with the nature of the Gumbel–Softmax distribution: smaller temperatures τ produce sharper, near-binary samples—one value dominates while others are near zero—enabling the model to effectively identify normality-relevant subgraphs, whereas a larger τ pushes the node soft weights toward 0.5, which reduces the discriminative ability of extracted subgraphs.

G. Time Efficiency (Q6)

As shown in Fig. 7, we compare the average training time per epoch (in seconds) of our proposed NGLAD with other GLAD variants across four real-world datasets. Among the methods, OCGIN, OCGTL, GLocalKD, and GOOD-D focus solely on anomaly detection without providing explanations, whereas SIGNET, GLADPro, and NGLAD can perform both detection and explanation. Methods that focus solely on distinguishing anomalous graphs from normal ones without providing explanations, such as OCGIN and GLocalKD, exhibit relatively faster training compared to SIGNET, GLADPro, and NGLAD, as expected. However, OCGTL is the slowest on Mutagen, MNIST, and REDDIT-BINARY, despite not producing explanations, due to its requirement to train multiple GNNs and perform transformation learning among them. GOOD-D also does not demonstrate a runtime advantage, since it performs hierarchical contrastive learning at the node, graph, and group levels for identifying anomalous graphs. Among the explanation-capable GLAD algorithms, including SIGNET and GLADPro, our proposed method NGLAD achieves superior efficiency on datasets Mutagen, REDDIT-BINARY, and COLLAB. Notably, REDDIT-BINARY and COLLAB are relatively larger social network datasets, where our method’s efficiency advantage is particularly pronounced.

VI. CONCLUSION

In this paper, we introduce the insight that normal graphs shared features, which manifest in key subgraphs referred to as normality-relevant subgraphs. These subgraphs are often absent or altered in anomalous graphs. Building on this observation, we propose a subgraph-based normality

learning approach to identify anomalous graphs and explain their identification based on the subgraphs they contain. To address the challenge of extracting key subgraphs that capture shared patterns in normal graphs, we present NGLAD. This method integrates a Subgraph Extractor and a Normality Learner within a one-class Information Bottleneck framework. Extensive evaluations on synthetic and real-world datasets demonstrate NGLAD’s superior performance in both anomaly detection and explanation.

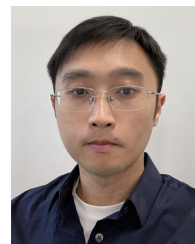
REFERENCES

- [1] C. C. Aggarwal and H. Wang, “Graph data management and mining: A survey of algorithms and applications,” *Manag. Mining Graph Data*, vol. 40, pp. 13–68, 2010.
- [2] K. Z. Khanam, G. Srivastava, and V. Mago, “The homophily principle in social network analysis: A survey,” *Multimedia Tools Appl.*, vol. 82, no. 6, pp. 8811–8854, Mar.2023.
- [3] T. Lanciano, F. Bonchi, and A. Gionis, “Explainable classification of brain networks via contrast subgraphs,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 3308–3318.
- [4] G. Zhang et al., “Dual-discriminative graph neural network for imbalanced graph-level anomaly detection,” in *Proc. NeurIPS*, 2022, pp. 24144–24157.
- [5] X. Ma et al., “A comprehensive survey on graph anomaly detection with deep learning,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12012–12038, Dec.2023.
- [6] C. Qiu, M. Kloft, S. Mandt, and M. Rudolph, “Raising the bar in graph-level anomaly detection,” in *Proc. Int. Joint Conf. Artif. Intell.*, Jul. 2022, pp. 2196–2203.
- [7] L. Zhao and L. Akoglu, “On using classification datasets to evaluate graph outlier detection: Peculiar observations and new insights,” *Big Data*, vol. 11, no. 3, pp. 151–180, Jun.2023.
- [8] R. Ma, G. Pang, L. Chen, and A. V. D. Hengel, “Deep graph-level anomaly detection by glocal knowledge distillation,” in *Proc. WSDM*, 2022, pp. 704–714.
- [9] L. Ruff et al., “Deep one-class classification,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [10] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” in *Proc. ICLR*, 2018.
- [11] S. Miao, M. Liu, and P. Li, “Interpretable and generalizable graph learning via stochastic attention mechanism,” in *Proc. ICML*, 2022, pp. 15524–15543.
- [12] Q. Sun et al., “Graph structure learning with variational information bottleneck,” in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2022, vol. 36, no. 4, pp. 4165–4174.
- [13] Y. Wu, X. Wang, A. Zhang, X. He, and T. Chua, “Discovering invariant rationales for graph neural networks,” in *Proc. ICLR*, 2022.
- [14] M. Zhang and P. Li, “Nested graph neural networks,” in *Proc. NeurIPS*, 2021, pp. 15734–15747.
- [15] L. Zhao, W. Jin, L. Akoglu, and N. Shah, “From stars to subgraphs: Uplifting any GNN with local structure awareness,” in *Proc. ICLR*, 2021.
- [16] Y. Liu, K. Ding, Q. Lu, F. Li, L. Y. Zhang, and S. Pan, “Towards self-interpretable graph-level anomaly detection,” in *Proc. NeurIPS*, 2023, pp. 8975–8987.
- [17] Z. Yang et al., “Global interpretable graph-level anomaly detection via prototype,” in *Proc. KDD*, 2025, pp. 3586–3597.
- [18] K. Crammer and G. Chechik, “A needle in a haystack: Local one-class optimization,” in *Proc. Int. Conf. Mach. Learn.*, 2004, p. 26.
- [19] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, “Spectral networks and locally connected networks on graphs,” in *Proc. ICLR*, 2013.
- [20] M. Defferrard, X. Bresson, and P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” in *Proc. NIPS*, vol. 29, 2016, pp. 3837–3845.
- [21] M. Balciar, G. Renton, P. Héroux, B. Gauzère, S. Adam, and P. Honein, “Analyzing the expressive power of graph neural networks in a spectral perspective,” in *Proc. ICLR*, 2021.
- [22] S. Sawlani, L. Zhao, and L. Akoglu, “Fast attributed graph embedding via density of states,” in *Proc. ICDM*, 2021, pp. 559–568.
- [23] T. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. ICLR*, 2016.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lió, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.

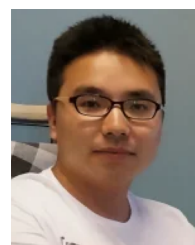
- [25] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, vol. 30, 2017, pp. 1024–1034.
- [26] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. ICLR*, 2018.
- [27] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An end-to-end deep learning architecture for graph classification," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 4438–4445.
- [28] Z. Wang and S. Ji, "Second-order pooling for graph neural networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 6, pp. 6870–6880, Jun. 2023.
- [29] Z. Ying, J. You, C. Morris, X. Ren, W. Hamilton, and J. Leskovec, "Hierarchical graph representation learning with differentiable pooling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4805–4815.
- [30] C. Morris et al., "Weisfeiler and Leman go neural: Higher-order graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, Jul. 2019, pp. 4602–4609.
- [31] H. Maron, H. Ben-Hamu, H. Serviansky, and Y. Lipman, "Provably powerful graph networks," in *Proc. NeurIPS*, vol. 32, 2019, pp. 2153–2164.
- [32] M. Balcilar, P. Héroux, B. Gauzere, P. Vasseur, S. Adam, and P. Honeine, "Breaking the limits of message passing graph neural networks," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, pp. 599–608.
- [33] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels," *J. Mach. Learn. Res.*, vol. 12, no. 77, pp. 2539–2561, 2011.
- [34] D. Luo et al., "Parameterized explainer for graph neural network," in *Proc. NIPS*, vol. 33, 2020, pp. 19620–19631.
- [35] H. Yuan, J. Tang, X. Hu, and S. Ji, "XGNN: Towards model-level explanations of graph neural networks," in *Proc. KDD*, 2020, pp. 430–438.
- [36] R. Henderson, D. Clevert, and F. Montanari, "Improving molecular graph neural network explainability with orthonormalization and induced sparsity," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 4203–4213.
- [37] M. N. Vu and M. T. Thai, "PGM-explainer: Probabilistic graphical model explanations for graph neural networks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 12225–12235.
- [38] J. Fang et al., "Evaluating post-hoc explanations for graph neural networks via robustness analysis," in *Proc. NeurIPS*, 2024, pp. 72446–72463.
- [39] T. H. A. Musa and A. Bouras, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 391–401, 2021.
- [40] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, no. 2, pp. 1–38, 2021.
- [41] Y. Sun, Y. Ming, X. Zhu, and Y. Li, "Out-of-distribution detection with deep nearest neighbors," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 20827–20840.
- [42] X. Lin et al., "Conformal graph-level out-of-distribution detection with adaptive data augmentation," in *Proc. WWW*, 2025, pp. 4755–4765.
- [43] G. Zhang, Z. Yang, J. Wu, P. Jiao, and J. Yang, "Enhancing graph neural networks for out-of-distribution graph detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 10, pp. 19255–19269, Oct. 2025.
- [44] B. Zong et al., "Deep autoencoding Gaussian mixture model for unsupervised anomaly detection," in *Proc. ICLR*, 2018.
- [45] N. Belton, M. T. Hagos, A. Lawlor, and K. M. Curran, "FewSOME: One-class few shot anomaly detection with Siamese networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2023, pp. 2978–2987.
- [46] G. Zhang et al., "FRAUDRE: Fraud detection dual-resistant to graph inconsistency and imbalance," in *Proc. ICDM*, 2021, pp. 867–876.
- [47] Q. Chen, S. Li, Y. Liu, S. Pan, C. Zhang, and Y. Zheng, "ARC: A generalist graph anomaly detector with in-context learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 50772–50804.
- [48] Z. Yuan, M. Shao, and Q. Yan, "Motif-level anomaly detection in dynamic graphs," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2870–2882, 2023.
- [49] L. Xi, R. Li, M. Li, D. Miao, R. Wang, and Z. J. Haas, "NMFAD: Neighbor-aware mask-filling attributed network anomaly detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 364–374, 2025.
- [50] X. Zhao et al., "Deep reinforcement learning guided graph neural networks for brain network analysis," *Neural Netw.*, vol. 154, pp. 56–67, Oct. 2022.
- [51] Y. Liu, K. Ding, H. Liu, and S. Pan, "GOOD-D: On unsupervised graph out-of-distribution detection," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, Feb. 2023, pp. 339–347.
- [52] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proc. ICLR*, 2016.
- [53] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. ICLR*, 2013.
- [54] T. Van Erven and P. Harremoës, "Rényi divergence and Kullback–Leibler divergence," *IEEE Trans. Inf. Theory*, vol. 60, no. 7, pp. 3797–3820, Jul. 2014.
- [55] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *J. Medicinal Chem.*, vol. 34, no. 2, pp. 786–797, Feb. 1991.
- [56] B. Knyazev, G. W. Taylor, and M. R. Amer, "Understanding attention and generalization in graph neural networks," in *Proc. NIPS*, 2019, pp. 4204–4214.
- [57] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, "TUDataset: A collection of benchmark datasets for learning with graphs," 2020, *arXiv:2007.08663*.
- [58] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Proc. NeurIPS*, 2019, pp. 9240–9251.
- [59] M. Schlichtkrull, N. D. Cao, and I. Titov, "Interpreting graph neural networks for NLP with differentiable edge masking," in *Proc. ICLR*, 2020.



Ge Zhang received the Ph.D. degree from the School of Computing, Macquarie University, Australia, in 2024. She is currently a Lecturer with the School of Information and Intelligent Science, Donghua University, China. She has published papers in prestigious journals and conferences, including IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, NeurIPS, WWW, IJCAI, ICDM, and WSDM. Her research focuses on graph anomaly detection.



Zhenyu Yang is currently pursuing the Ph.D. degree with the School of Computing, Macquarie University, Australia. His current research interests include deep learning and graph learning.



Jia Wu (Senior Member, IEEE) received the Ph.D. degree from the University of Technology Sydney, Australia. He is currently an ARC DECRA Fellow and a Senior Lecturer (Associate Professor in North America) with the School of Computing, Macquarie University, Australia. His current research interests include data mining and machine learning. He is an Associate Editor of IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS (TNNLS).



Pengfei Jiao (Member, IEEE) received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2018. From 2018 to 2021, he was a Lecturer with the Center of Biosafety Research and Strategy, Tianjin University. He is currently a Professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His current research interests include complex network analysis and its applications.



Hao Peng (Senior Member, IEEE) is currently a Professor with the School of Cyber Science and Technology, Beihang University. His current research interests include machine learning, deep learning, and reinforcement learning. He is an Associate Editor of *International Journal of Machine Learning and Cybernetics*(IJMLC) and *Neural Networks*.



Jian Yang (Member, IEEE) received the Ph.D. degree in data integration from The Australian National University, Canberra, ACT, Australia, in 1995. She is currently a Full Professor with the School of Computing, Macquarie University, Sydney, NSW, Australia. Her research interests include business process management, data science, and social networks. She is currently serving as an Executive Committee for the Computing Research and Education Association of Australasia.



Xixun Lin received the M.S. degree from the College of Computer Sciences and Technology, Jilin University, in 2018. He is currently pursuing the Ph.D. degree in computer science with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include information retrieval, recommender system, and graph representation learning. He has published several articles in IEEE TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE, ICML, WWW, ICDM, WSDM and SIGIR.